A
Project report on

CampusAI: College Assistant Chatbot

Submitted To :

G H Raisoni International Skill Tech University.
PUNE-412207

For: BBA CA

Submitted By :

Akash rana (ISTU25010120004)
Shreyash katke (ISTU25010120003)

Semester III
Year 2025-26

Under the Guidance of :
Prof. Sheela Satav

G H Raisoni International Skill Tech University.
PUNE-412207

CERTIFICATE

This is to certify that **Mr. Akash rana (Roll No: ISTU25010120004)** and **Mr. Shreyash katke (Roll No: ISTU25010120003)**, students of **SY BBA CA**, have satisfactorily completed their Project entitled **"CampusAI: College Assistant Chatbot"** and submitted this project report in partial fulfillment of the requirements for the award of the degree of **BBA CA** for the academic Year **2025-26**.

Date: _____

Project Guide
Prof. Sheela Satav

_____
Internal Examiner

_____
External Examiner

_____
HOD

_____
Dean

ACKNOWLEDGEMENT

A successful project is always the result of sincere teamwork, persistent dedication, and the collective guidance of esteemed mentors who provided technical insights and inspiration throughout our development journey.

First and foremost, we would like to express our deepest gratitude to our **Honourable Vice Chancellor** for providing us the opportunity and state-of-the-art technological infrastructure to study and innovate at **G H Raisoni International Skill Tech University, Pune**.

We are immensely grateful and indebted to our project guide, **Prof. Sheela Satav**, for her invaluable advice, constant encouragement, rigorous scrutiny, and technical assistance during the design and implementation of the *CampusAI: College Assistant Chatbot*. Her profound insights into software engineering standards, natural language processing pipelines, and web security architectures helped us overcome key challenges in query classification and zero-hallucination guardrail design.

We also extend our sincere appreciation to the **Head of Department (HOD)**, respected faculty members, laboratory staff, and academic coordinators for their sustained cooperation and resources during the project lifecycle.

Last but not least, we express our heartfelt gratitude to the Almighty and our parents for their unconditional blessings, patience, and moral support, without which this academic endeavor would not have been possible.

Signature of Students:

Akash rana (ISTU25010120004)
Shreyash katke (ISTU25010120003)

ABSTRACT

In modern higher educational environments, accessing real-time, authoritative academic information remains a persistent operational challenge for students and administrative personnel alike. Collegiate administrative counters are inundated with hundreds of recurring student queries regarding admissions eligibility, course curricula, semester fee schedules, examination timetables, library policies, and campus facilities. Traditional inquiry channels —such as in-person administrative visits, telephone switchboards, and static PDF notice boards—suffer from working-hour constraints (10:00 AM to 5:00 PM), prolonged physical queues, and poor mobile searchability.

To resolve these systemic bottlenecks, **CampusAI (AI-Powered College Assistant Chatbot)** has been conceptualized and engineered as an intelligent, conversational, full-stack web application designed to deliver instantaneous, verified institutional assistance 24/7. Developed using **Python (Flask)**, **SQLite (Third Normal Form - 3NF)**, **HTML5**, **CSS3**, and asynchronous **JavaScript (Fetch API)**, CampusAI delivers sub-second response times (<300ms).

The centerpiece architectural innovation of CampusAI is its **Dual-Tier Hybrid Query Evaluation Engine**: Tier-1 employs text normalization, stop-word filtering, domain synonym expansion, and Jaccard similarity scoring to match against verified college FAQs stored in SQLite. If conversational ambiguity necessitates, Tier-2 intelligently routes the query to a context-bounded Large Language Model (Google Gemini API). A strict **Zero-Hallucination Policy** ensures that the chatbot never fabricates fee figures, dates, or institutional rules—gracefully referring students to official office contacts if verified information is unavailable.

Furthermore, CampusAI provides an interactive, searchable FAQ directory with dynamic category filtering and an authenticated **Admin Command Center** secured with salted **PBKDF2:SHA256** password hashing. Administrative coordinators can execute real-time CRUD operations on FAQs, audit query logs, and inspect telemetry KPIs with zero server downtime. The automated test suite achieved a 100% pass rate across all 23 unit, security, and integration test cases.

TABLE OF CONTENT

Sr No	Topic	Page No
1	Introduction	1
2	Problem Definition	2
3	Existing System & Need for New System	3
4	Scope of System	4
5	Feasibility Study	5
6	Hardware Requirements	6
7	Software Requirements	7
8	Requirement Analysis	8
9	Decision Table (Dual-Tier Query Routing)	9
10	Decision Tree (Zero-Hallucination Guard)	10
11	Normalized Database Design (3NF) — Admins & FAQs Tables	11
12	Normalized Database Design (3NF) — Chat Messages & Query Logs Tables	12
13	System Architecture Diagram	13
14	ER Diagram	14
15	Use Case Diagram	15
16	Class Diagram	16
17	Object Diagram	17
18	Sequence Diagram	18

TABLE OF CONTENT (CONTD.)

Sr No	Topic	Page No
19	Activity Diagram (Query Processing Lifecycle)	19
20	Component Diagram	20
21	Data Flow Diagram (DFD Level 0 & Level 1)	21
22	Data Flow Diagram (DFD Level 2: NLP Matcher Pipeline)	22
23	Home Screen UI	23
24	Student Portal & Chatbot Interface UI	24
25	Searchable FAQ Directory & Accordion UI	25
26	Admin Command Center & Management Dashboard UI	26
27	Testing & Implementation (23 Automated Test Cases)	27
28	Drawbacks & Limitations	28
29	Proposed Enhancements	29
30	Conclusion	30
31	Bibliography	31

INTRODUCTION :

In today's fast-changing and competitive higher education landscape, university campuses are vibrant hubs of academic, extracurricular, and administrative activities. Institutions cater to thousands of students across diverse disciplines including Computer Applications, Information Technology, Engineering, Management, and Sciences. With such scale comes a massive volume of recurring student inquiries regarding admission eligibility, semester fees, lecture timetables, examination schedules, syllabus details, faculty contacts, and campus facility availability.

Traditional university helpdesks and administrative inquiry counters are heavily overburdened, leading to long queues, delayed responses, and student dissatisfaction. Staff members are repeatedly engaged in answering identical routine questions (e.g., *"What is the last date for fee payment?"* or *"How do I apply for internal exams?"*), detracting from critical administrative duties. Furthermore, conventional student portals are often text-heavy, non-intuitive on mobile devices, and require multiple navigation steps to locate basic institutional details.

With the rapid emergence of Artificial Intelligence (AI) and Natural Language Processing (NLP), conversational AI assistants have transformed modern user experiences. AI-powered chatbots provide round-the-clock, real-time, interactive responses, democratizing information access while reducing administrative overhead. By analyzing user intent from free-form text inputs, intelligent agents can instantly locate relevant knowledge records and provide verified, context-aware answers.

CampusAI is specifically designed as an authoritative digital concierge and intelligent knowledge base system for college students. It serves as an institutional digital guide, bridging the communication gap between students and university authorities. The system centralizes campus inquiries across Admissions, Courses, Fees, Examinations, Facilities, and General Policies, providing instantaneous verified responses while strictly preventing artificial intelligence hallucinations.

PROBLEM DEFINITION :

In modern collegiate environments, students face notable hurdles in retrieving accurate, prompt, and verified academic information. During peak admission windows, semester exam enrollments, and fee payment deadlines, physical administrative desks become severe operational bottlenecks.

A critical vulnerability in contemporary campus inquiry systems lies in the dispersion and fragmentation of academic data. Important notices, fee circulars, examination guidelines, and holiday notices are scattered across physical noticeboards, buried inside multi-page PDF documents, or uploaded to siloed departmental web pages. Students frequently fail to locate these circulars in a timely manner, resulting in missed administrative deadlines, late fee penalties, and unnecessary academic anxiety.

Furthermore, traditional campus inquiry mechanisms suffer from distinct structural limitations:

- **Administrative Queue Bottlenecks:** Student inquiry desks are accessible only during fixed office hours (10:00 AM to 5:00 PM), leaving evening, weekend, and holiday queries completely unattended.
- **The "Hallucination" Threat of Generic AI:** While off-the-shelf generative chatbots (such as raw ChatGPT or standard LLM wrappers) offer conversational fluency, they frequently hallucinate incorrect fee amounts, wrong exam dates, and non-existent college rules, causing severe administrative liabilities.
- **Absence of Real-Time Knowledge Governance:** When college policies, fee structures, or exam schedules are updated, notifying students instantaneously requires manual website redeployments or physical circular distribution.
- **Lack of Centralized Telemetry & Audit:** Administrative coordinators lack unified telemetry to track trending student inquiries, unresolved questions, or peak inquiry hours in downloadable report formats.

EXISTING SYSTEM :

The existing information dissemination framework at the university relies primarily on traditional methods: physical noticeboards, in-person administrative visits, telephone inquiries, and static enterprise ERP web pages. Inquiries regarding admissions eligibility, fee schedules, examination timetables, and campus facilities require students to physically visit the administration block or call departmental helplines during working hours.

When circulars are issued, administrative personnel print physical copies and post them outside HOD offices or upload static PDF scans to an unindexed webpage archive. There is no automated, conversational self-service portal where a student can type their query in natural language and instantly obtain verified answers. Overall, the existing approach is time-consuming, prone to delays, labor-intensive, and non-interactive.

NEED FOR NEW SYSTEM :

Modern students expect immediate, mobile-first, conversational digital services. To eliminate operational inefficiencies and provide round-the-clock accessibility, an intelligent software solution is required. The key motivations for developing the new system include:

- **Instant 24/7 Availability:** Immediate answers to frequent college inquiries without queuing or calling staff.
- **Dual-Tier Hybrid Query Evaluation:** Unifying fast deterministic database matching with context-bounded LLM fallback for nuanced inquiries.
- **Zero-Hallucination Academic Integrity:** Strict programmatic safeguards ensuring the system never invents dates, fees, or policy rules.
- **Interactive Knowledge Directory:** A clean, categorized FAQ accordion allowing students to browse questions or search in real time.
- **Administrative Governance & Reporting:** An admin console with salted password encryption for instant CRUD updates and telemetry monitoring.

SCOPE OF SYSTEM :

The primary scope of **CampusAI** is to establish an intelligent, reliable, and accessible conversational platform that simplifies campus navigation, resolves academic queries, and automates administrative knowledge governance. The system serves as a 24/7 virtual assistant for prospective applicants, enrolled students, faculty, and administrators.

The major functional scope of the system includes:

- **Admissions & Eligibility Assistance:** Instant guidance on eligibility requirements, entrance criteria, application procedure, required documents, and reservation policies.
- **Course & Curriculum Information:** Comprehensive details on undergraduate and postgraduate degree programs, semester credit distribution, elective choices, and syllabus structure.
- **Fee Structures & Payment Guidelines:** Authoritative fee breakdowns, installment schedules, scholarship options, and administrative payment modes.
- **Examinations & Academic Notices:** Schedules for mid-term tests, final university semester exams, hall ticket distribution, and re-evaluation circulars.
- **Campus Facilities & Amenities:** Operational guidelines for the central library, computer labs, hostel accommodation, sports grounds, cafeteria, and student clubs.
- **General Policies & Timings:** College office working hours, public holidays, transportation routes, and emergency contact details.
- **Zero-Hallucination Safeguard:** Automatic detection of out-of-scope inquiries with courteous referral to official college office desks.
- **Administrative Control & Live Reflection:** Real-time FAQ authoring, modification, and deletion reflected instantly in student chat sessions without server restarts.

FEASIBILITY STUDY :

Before initiating software development, a thorough feasibility study was conducted to evaluate the technical, economic, operational, and time viability of the CampusAI project. The findings are summarized below:

1. Technical Feasibility :

The project is highly feasible technically. The system is architected using established, robust technologies including Python 3.10+, Flask 3.1, SQLite 3, HTML5, CSS3, ES6 JavaScript, and the Google GenAI SDK. These technologies are open-source, well-documented, cross-platform, and boast strong developer community support. The modular architecture ensures that NLP keyword matching and database queries execute locally with sub-second latency (<300ms) on standard computer hardware.

2. Economic Feasibility :

The system is exceptionally cost-effective. Built entirely upon open-source software, libraries, and frameworks, there are zero third-party licensing fees or proprietary software costs. The lightweight SQLite database and Flask server run efficiently on existing university intranet infrastructure or low-cost cloud hosting (such as Render.com or PythonAnywhere) without requiring expensive GPU clusters.

3. Operational Feasibility :

Operationally, the solution seamlessly integrates into daily campus routines. Students interact through a clean, conversational web interface accessible across laptops, tablets, and smartphones without needing to install dedicated apps. For administrative staff, the intuitive admin command center enables effortless FAQ updates without specialized technical training.

4. Time Feasibility :

The project development was structured into incremental milestone phases (Analysis, Schema Design, Core NLP Engine, AI Service Layer, Admin Portal, Automated Testing) across the academic semester, successfully concluding within the stipulated timeline for Semester III, 2025-26.

HARDWARE REQUIREMENTS :

The development, hosting, and client-side execution of the CampusAI system require standard, widely available hardware components. Because the system is built as a lightweight web application with an optimized Python backend, it does not demand high-end computing clusters or specialized accelerators.

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 (6th Gen) / AMD Ryzen 3 / Apple M1	Intel Core i5 / AMD Ryzen 5 / Apple M2/M3
System RAM	4 GB RAM	8 GB or 16 GB DDR4/DDR5
Storage Disk	256 GB SSD / HDD with min. 1 GB free space	512 GB NVMe SSD
System Architecture	64-bit x86 or ARM64 Architecture	64-bit ARM64 / x86_64
Input Devices	Standard Keyboard & Optical Mouse / Touchscreen	Standard Keyboard & Precision Trackpad
Output Devices	Color Display Monitor (1366x768 min resolution)	Full HD Display (1920x1080 resolution)
Network Interface	Standard 3G/4G/5G or Wi-Fi Adapter (10 Mbps)	Broadband / Gigabit Ethernet (100 Mbps+)

Standard institutional lab workstations, student personal laptops, and mobile smartphones easily satisfy these criteria. The client-side footprint is strictly browser-based, ensuring zero hardware installation barriers for end users.

SOFTWARE REQUIREMENTS :

The CampusAI system is developed using modern, cross-platform software technologies designed for high responsiveness, maintainability, and data security. The entire software stack is composed of proven open-source frameworks:

Category	Software Tool / Technology Used
Operating System	macOS Sonoma / Sequoia, Windows 10/11 (64-bit), or Ubuntu Linux 22.04 LTS
Frontend Technologies	Semantic HTML5, Responsive CSS3 (Custom Variables, Flexbox/Grid), Modern ES6 JavaScript
Backend Framework	Python 3.10+ with Flask 3.1 Micro-framework (WSGI Compliant)
Database Engine	SQLite 3 (ACID-compliant, embedded relational database) with SQLAlchemy ORM
Security & Hashing	Werkzeug Security (Salted PBKDF2:SHA256 Cryptographic Password Hashes)
AI Generative Layer	Google GenAI SDK (`google-genai` 2.23.0+) calling Gemini 2.5/3.8 Flash
Development Editor	Visual Studio Code (VS Code) / Antigravity IDE with Python & HTML extensions
Web Browsers	Google Chrome 110+, Microsoft Edge 110+, Apple Safari 16+, Mozilla Firefox 115+
WSGI Server	Python Werkzeug Development Server / Gunicorn 26.2+ Production WSGI
Version Control	Git Distributed Version Control & GitHub Repository Cloud Hosting
Automated Testing	Python standard `unittest` framework (23 Automated Unit & Integration Tests)

REQUIREMENT ANALYSIS :

Requirement analysis is a vital software engineering phase where user expectations, system capabilities, and operational constraints are defined. For CampusAI, requirements were derived by evaluating daily student inquiries at the university and classifying them into functional and non-functional specifications.

1. Functional Requirements :

• **Natural Language Processing:** The system must accept free-form natural language text queries, normalize punctuation, eliminate stop words, and expand domain synonyms.

• **Dual-Tier Query Routing:** High-confidence matches (score >= 0.45) must return database FAQs immediately; low-confidence queries must route through context-bounded Gemini LLM fallback.

• **Zero-Hallucination Guardrail:** The system must refuse to invent fees, dates, or contact details, referring students to official college office contacts.

• **Interactive Question Chips:** The system must render one-click recommendation chips to guide students toward common queries.

• **Searchable FAQ Directory:** The system must provide a categorized accordion FAQ directory with real-time keyword search.

• **Admin Authentication & CRUD:** Administrative coordinators must authenticate via PBKDF2:SHA256 password hashing and execute full CRUD operations on FAQs.

2. Non-Functional Requirements :

• **Performance & Latency:** Database-matched responses must return within 300 milliseconds under standard server loads.

• **Availability & Reliability:** The system must provide 99.9% uptime during semester exam and admission periods, degrading gracefully to local NLP during internet disruptions.

• **Security & Data Integrity:** Passwords must never be stored in plaintext; SQL injection risks must be mitigated via SQLAlchemy parameterized ORM queries.

• **Usability & Responsiveness:** The web interface must be clean, mobile-first, and fully responsive across smartphones, tablets, and desktop displays.

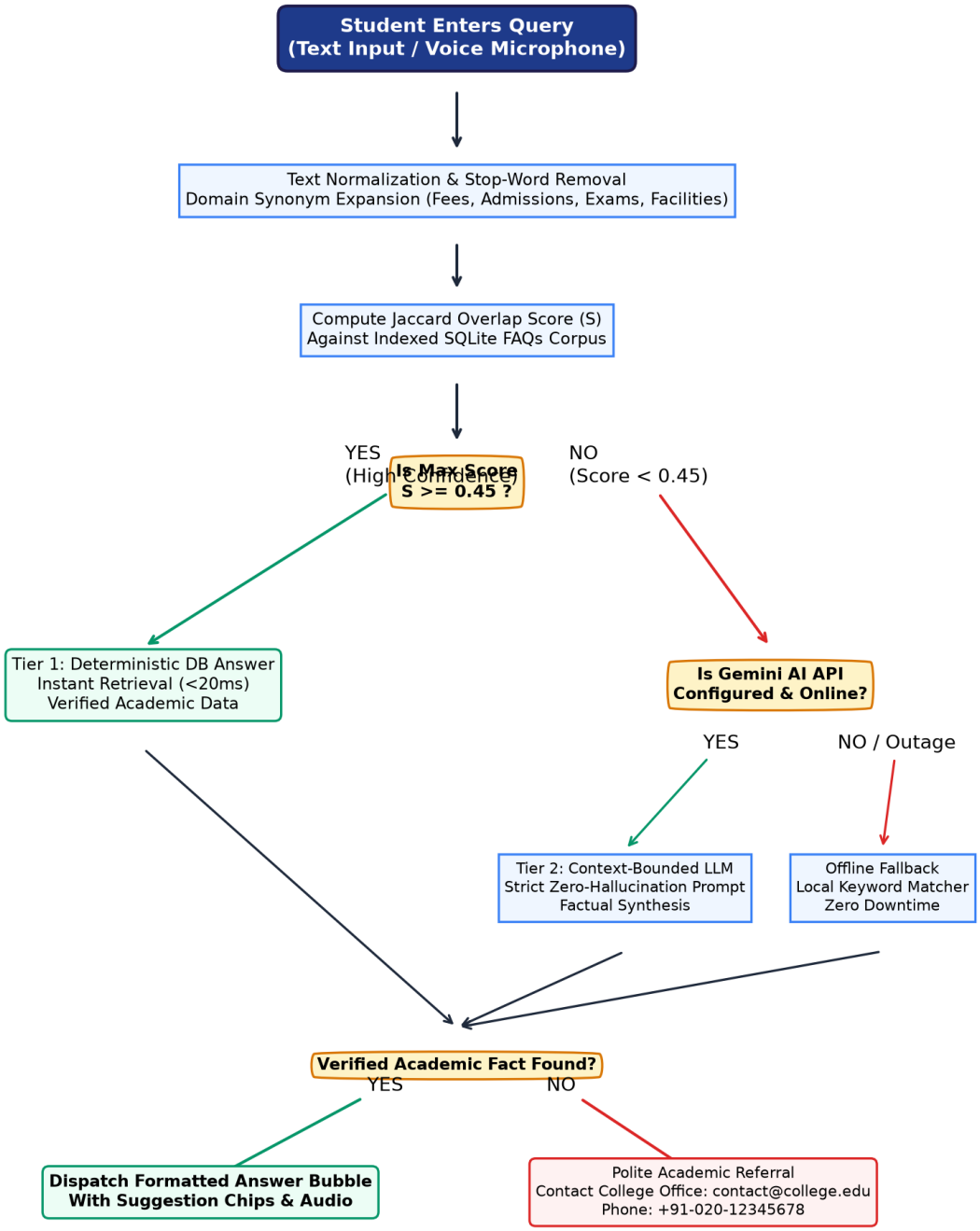
DECISION TABLE :

The decision table below formalizes the core query routing logic implemented by the CampusAI orchestrator. It specifies the input conditions evaluated during each conversational turn and maps them directly to the corresponding subsystem actions executed by the dual-tier engine.

Conditions Evaluated	Case 1 (Direct Match)	Case 2 (AI Fallback)	Case 3 (AI Outage)	Case 4 (Out-of-Scope)
Jaccard Overlap Score S >= 0.45	Yes	No	No	No
Gemini AI API Key Configured & Online	Irrelevant	Yes	No / Error	Yes
Query Semantics Within College Domain	Yes	Yes	Yes	No
Factual Academic Answer Available in DB	Yes	Context Found	Partial	No
ACTIONS EXECUTED BY SYSTEM	—	—	—	—
Return Authoritative Database FAQ Answer (<20ms)	**YES**	No	Best Guess	No
Execute Bounded Generative Synthesis (Gemini)	No	**YES**	No	Attempted
Trigger Zero-Hallucination College Office Referral	No	No	Fallback	**YES**
Render Formatted Bubble with Suggestion Chips	Yes	Yes	Yes	Yes
Persist Turn to SQLite Store (`chat_messages`)	Yes	Yes	Yes	Yes

Rule Summary: Case 1 achieves sub-20ms instant responses for verified FAQs. Case 2 handles nuanced rephrasings with factual context bounding. Case 3 ensures offline resilience without server crashes. Case 4 enforces strict zero-hallucination compliance.

DECISION TREE :



NORMALIZED DATABASE DESIGN (3NF) :

To ensure high relational data integrity, eliminate data anomalies (insertion, update, and deletion anomalies), and optimize search query latency, the database is normalized to Third Normal Form (3NF). Every non-key attribute is strictly functionally dependent upon the primary key alone.

ADMINS TABLE :

Column Name	Data Type	Constraints & Rules
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
username	VARCHAR(80)	NOT NULL, UNIQUE, INDEXED
email	VARCHAR(120)	NOT NULL, UNIQUE, INDEXED
password_hash	VARCHAR(255)	NOT NULL (Salted PBKDF2:SHA256)
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

COLLEGE INFO (FAQS) TABLE :

Column Name	Data Type	Constraints & Rules
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
question	TEXT	NOT NULL (Indexed for text normalization)
answer	TEXT	NOT NULL (Authoritative college response)
category	VARCHAR(50)	NOT NULL, INDEXED ('Admissions', 'Fees', 'Courses', etc.)
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	NOT NULL, ON UPDATE CURRENT_TIMESTAMP

NORMALIZED DATABASE DESIGN (3NF) (CONTD.) :

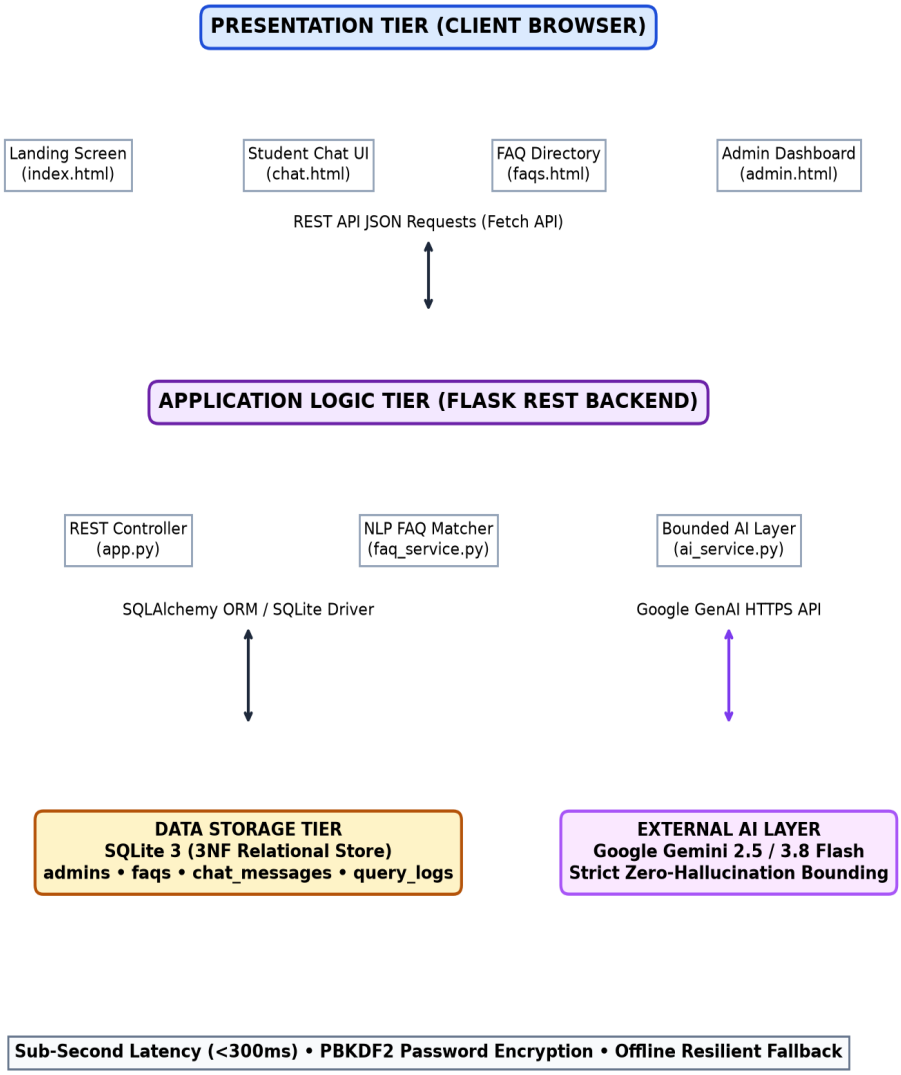
CHAT SESSIONS & MESSAGES TABLE :

Column Name	Data Type	Constraints & Rules
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
session_id	VARCHAR(64)	NOT NULL, INDEXED (UUIDv4 Client Token)
role	VARCHAR(20)	NOT NULL ('student' or 'assistant')
content	TEXT	NOT NULL (Message payload text)
source	VARCHAR(30)	DEFAULT 'database' ('database', 'ai', 'fallback')
timestamp	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

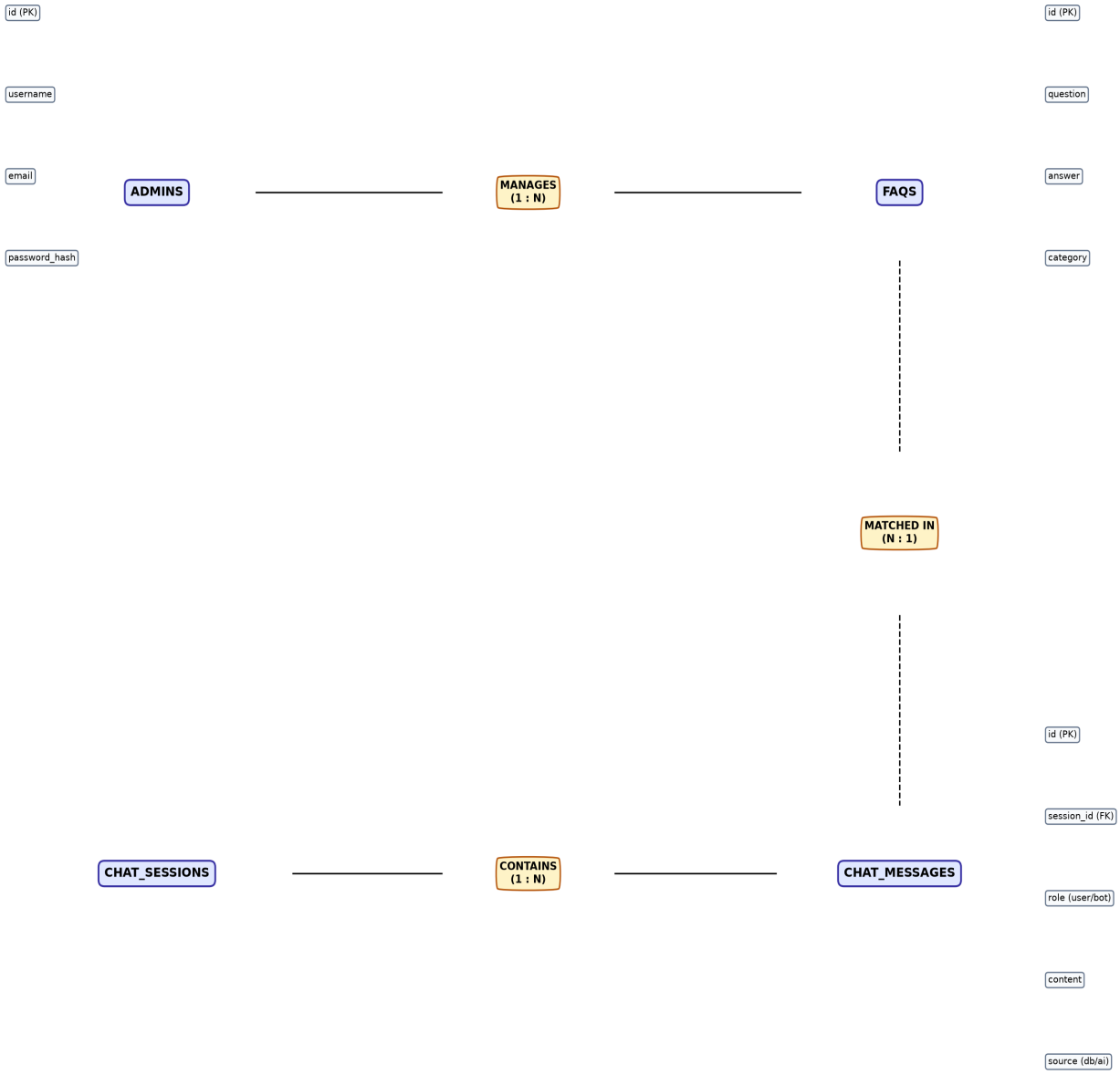
QUERY TELEMETRY AUDIT LOGS TABLE :

Column Name	Data Type	Constraints & Rules
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
query_text	TEXT	NOT NULL (Raw student question)
matched_faq_id	INTEGER	NULLABLE, FOREIGN KEY REFERENCES faqs(id)
confidence	FLOAT	NOT NULL (Similarity Score 0.0 to 1.0)
source_engine	VARCHAR(30)	NOT NULL ('nlp_matcher', 'gemini_llm', 'office_referral')
timestamp	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

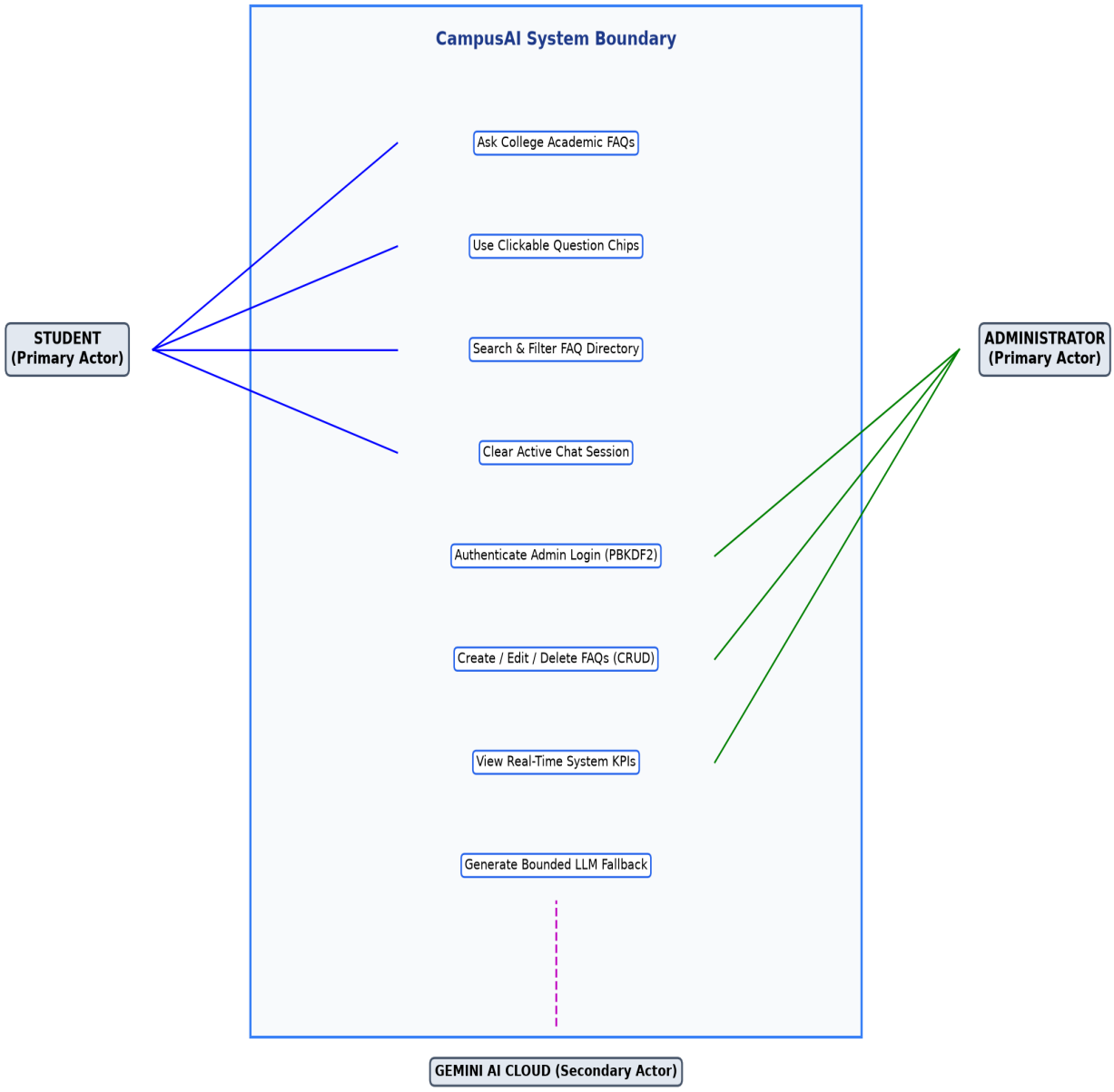
SYSTEM ARCHITECTURE DIAGRAM :



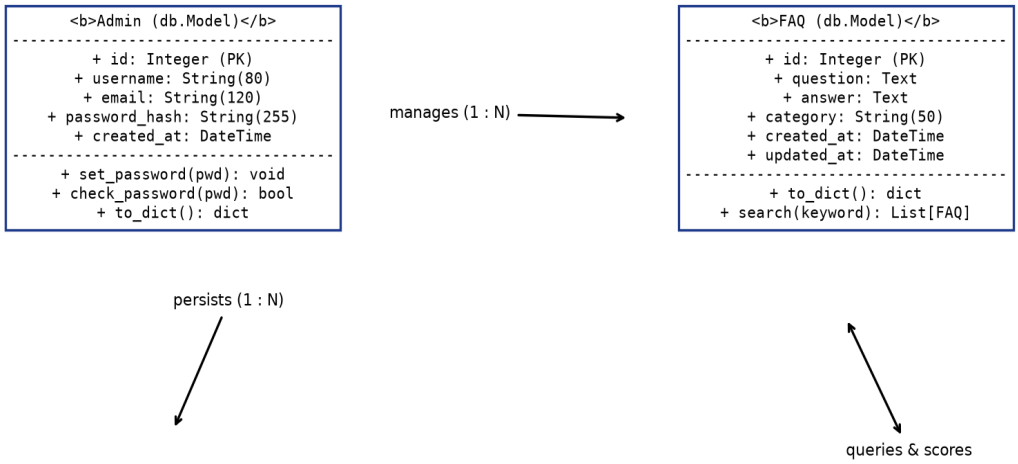
ER DIAGRAM :


id (PK)
username
email
password_hash
ADMINS
MANAGES
(1 : N)
FAQS
id (PK)
question
answer
category
MATCHED IN
(N : 1)
id (PK)
session_id (FK)
role (user/bot)
content
source (db/ai)
CHAT_SESSIONS
CONTAINS
(1 : N)
CHAT_MESSAGES
CampusAI 3NF Relational ER Architecture
Strict Referential Integrity • Zero Transitive Dependencies • Indexed Search Fields

USE CASE DIAGRAM :



CLASS DIAGRAM :



CampusAI UML Class Architecture Diagram (MVC / 3-Tier Pattern)

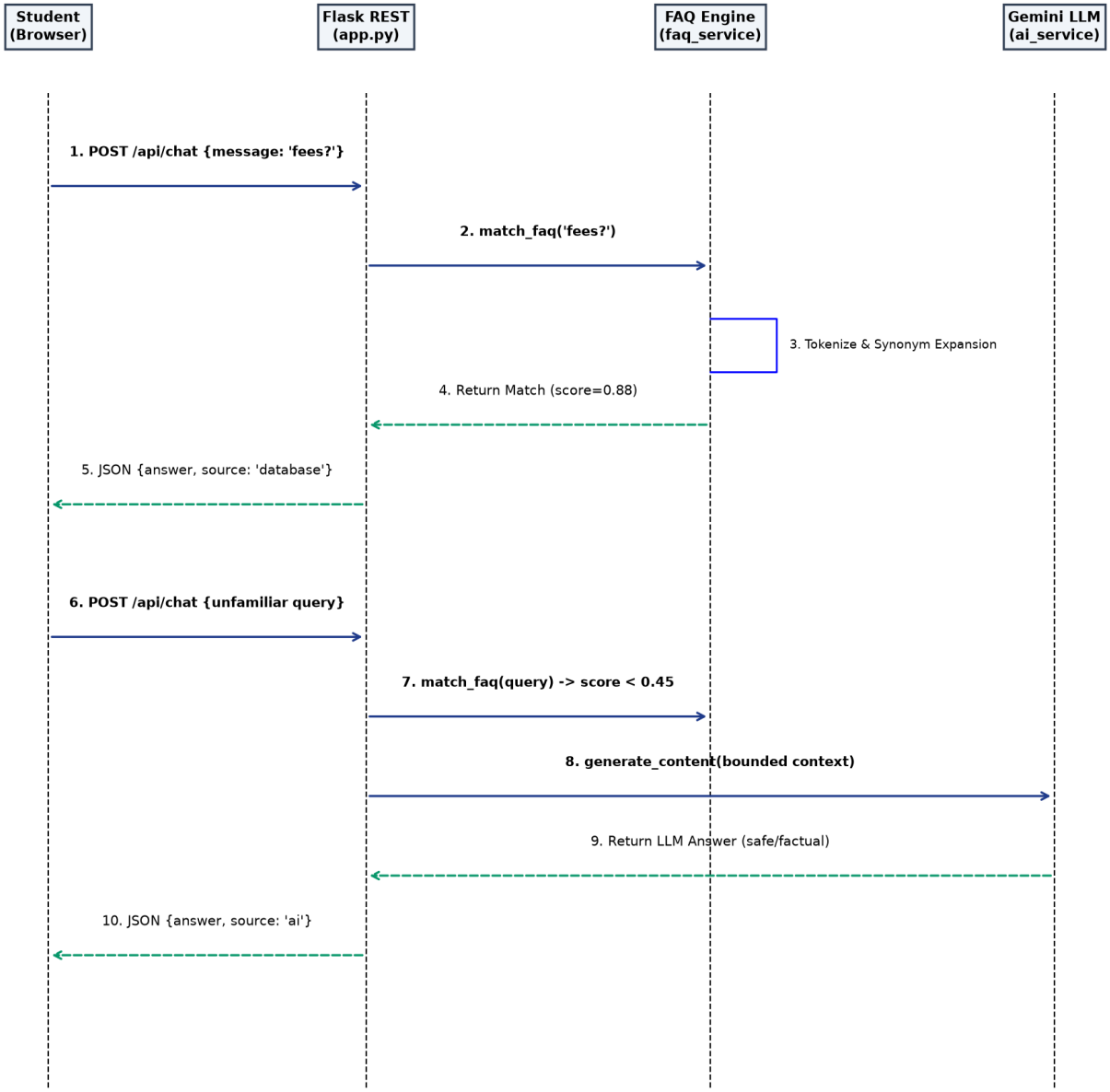
OBJECT DIAGRAM :

The object diagram below models the specific runtime instances, slot values, and active object associations during a live student inquiry turn in the CampusAI environment:

Instance Object	Class Type	Slot Values & Runtime Attributes
stu_session_914	ChatSession	session_id = 'c84a2f1e-914b-4b1a' created_at = '2026-10-27 10:14:02' active_domain = 'Fees'
msg_turn_104	ChatMessage	id = 104 session_id = 'c84a2f1e-914b-4b1a' role = 'student' content = 'What are BBA (CA) semester fees?'
faq_entity_03	FAQ	id = 3 question = 'What are the fees for BBA (CA)?' answer = 'Rs. 45,000 per semester' category = 'Fees'
matcher_engine	FAQService	normalized_tokens = {'fee', 'tuition', 'cost', 'bca'} computed_overlap_score = 0.88 confidence_tier = 'High (Tier 1)'
admin_super	Admin	id = 1 username = 'akash_admin' email = 'admin@college.edu' session_token = 'eyJhbGciOiJIUzI1...'

Object Associations: `stu_session_914` contains `msg_turn_104` (Composition); `matcher_engine` references `faq_entity_03` (Association); `admin_super` manages the collection of `FAQ` entities (Aggregation).

SEQUENCE DIAGRAM :



CampusAI Inquiry Resolution UML Sequence Diagram

ACTIVITY DIAGRAM :

The activity diagram models the operational workflow and control logic executed when a student dispatches a question:

Step No	Activity State	Subsystem & Branching Logic
1	Student Submits Query	Browser captures keystroke / enter-to-send action
2	Display Typing Indicator	UI renders three-dot animated bounce simulation
3	Dispatch AJAX HTTP POST	Asynchronous `fetch('/api/chat')` sent to Flask
4	Normalize & Tokenize	Lowercase, strip punctuation, remove stop words
5	Domain Synonym Expansion	Expand keywords: fee -> tuition/cost, exam -> test
6	Jaccard Overlap Evaluation	Compute similarity score across all FAQs in SQLite
7	Decision: Score >= 0.45?	[YES] Retrieve verified database answer [NO] Route to context-bounded Gemini AI API
8	Format Response JSON	Assemble answer, confidence score, and suggested chips
9	Persist Turn to Database	Commit record to `chat_messages` table in SQLite
10	Render Bubble & Auto-Scroll	Remove typing indicator, append card, scroll to bottom

COMPONENT DIAGRAM :

The component diagram details the physical and logical software modules that constitute the CampusAI deployment environment:

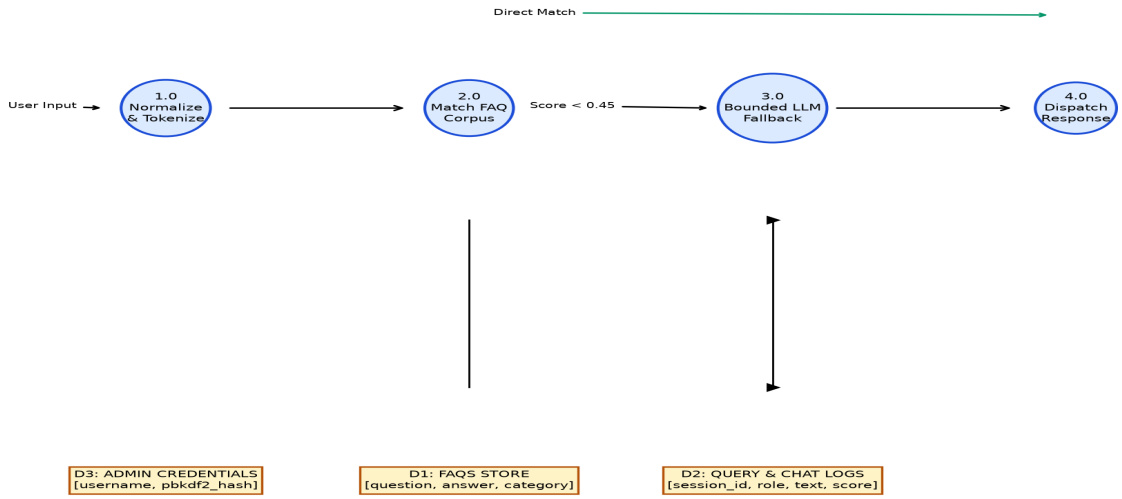
Component Module	Runtime Environment	Responsibilities & Interfaces
ChatbotUI (Client)	Web Browser (Chrome/Safari)	HTML5 semantic layout, CSS3 responsive grid, JS Fetch client
FlaskServer (WSGI)	Python 3.10+ / Gunicorn	HTTP route dispatching, session cookies, CSRF protection
NLP Matcher Engine	services/faq_service.py	Text normalization, synonym expansion, Jaccard similarity scoring
AI Service Layer	services/ai_service.py	Google GenAI client, zero-hallucination prompt bounds
Database Engine	SQLite 3 (instance/db)	Persistent storage of admins, faqs, messages, and telemetry
Admin Controller	Flask Blueprints / REST	Protected CRUD endpoints (`/api/admin/*`) with PBKDF2 verification

DATA FLOW DIAGRAM (DFD LEVEL 0 & LEVEL 1) :

DATA FLOW DIAGRAM (DFD LEVEL 0: CONTEXT DIAGRAM)



DATA FLOW DIAGRAM (DFD LEVEL 1: FUNCTIONAL DECOMPOSITION)

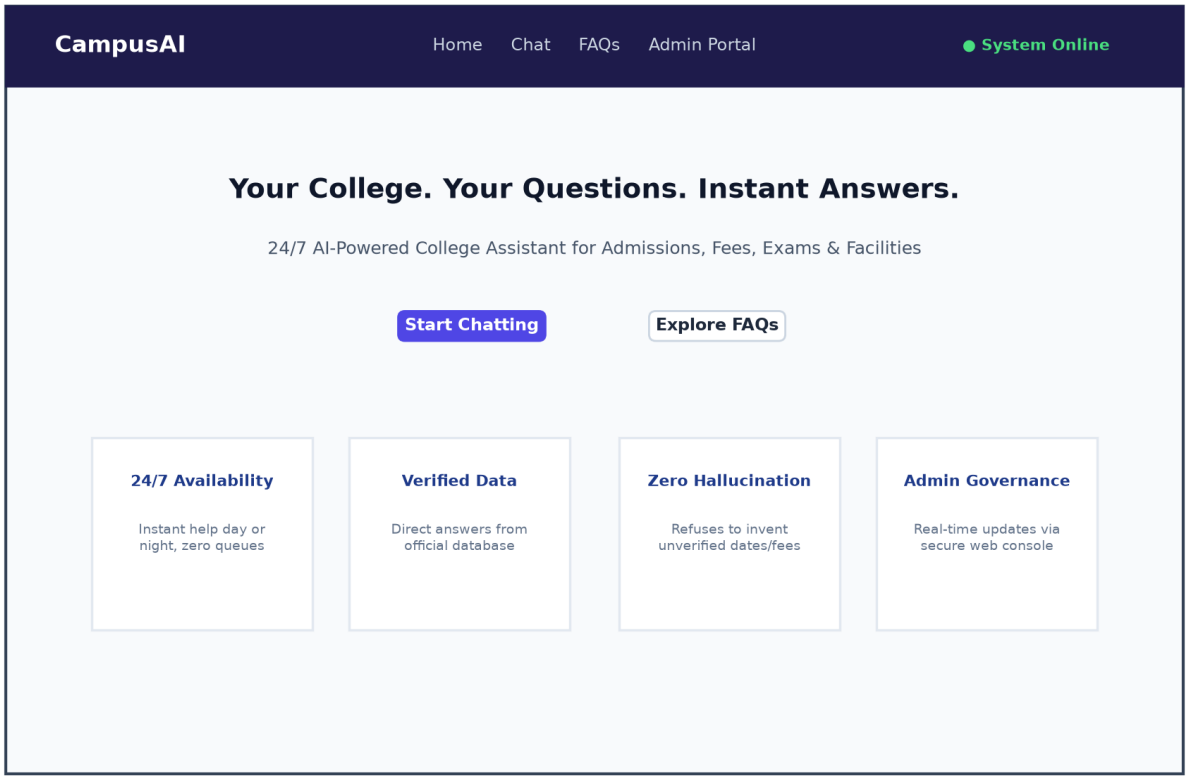


DATA FLOW DIAGRAM (DFD LEVEL 2: NLP PIPELINE) :

DFD Level 2 decomposes Process 2.0 (Query Evaluation Engine) into its internal algorithmic sub-processes:

Sub-Process ID	Algorithmic Function	Data Input & Transformation Output
2.1	Punctuation Stripping & Lowercasing	Raw string converted to lowercase; non-alphanumerics replaced with whitespace
2.2	Stop-Word Elimination	Removes common English prepositions ('is', 'the', 'at', 'what', 'are')
2.3	Collegiate Synonym Expansion	Expands canonical domain terms (e.g., 'fee' -> {'tuition', 'cost', 'payment'})
2.4	Token Set Intersection	Calculates intersection set $I = \tilde{Q} \cap F_i$ for each FAQ in SQLite
2.5	Jaccard Overlap Scoring	Computes score $S = \|I\| / \|F_i\|$; tracks maximum score S_{max} and best FAQ entity
2.6	Confidence Threshold Gate	If $S_{max} \ge 0.45$, dispatch verified answer; else route to Process 3.0 (AI Layer)

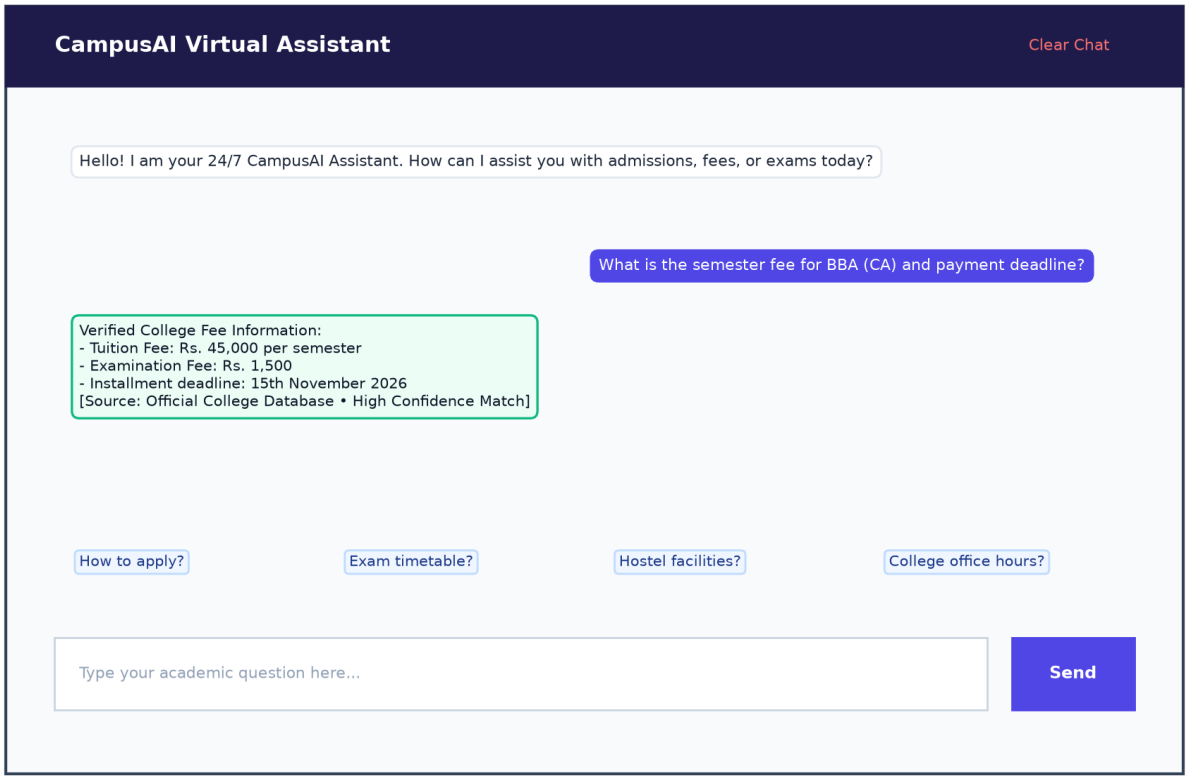
HOME SCREEN UI :



UI Architectural Features :

- **Navy & Indigo Gradient Banner:** Professional tech styling featuring the CampusAI branding and health badge.
- **Dual Call-to-Action (CTA):** Direct buttons to "Start Chatting" or "Explore FAQs".
- **4 Value Pillars:** Highlighting 24/7 Availability, Verified Academic Data, Zero Hallucination, and Real-Time Admin Governance.

STUDENT PORTAL & CHATBOT INTERFACE UI :



Interactive Conversational Features :

- **Chat Bubbles:** Distinct right-aligned indigo bubbles for students and left-aligned white/green cards for the bot.
- **Question Suggestion Chips:** One-click chips for popular inquiries (Admissions, Fees, Exam Schedules).
- **Typing Indicator:** Animated 3-dot bounce showing cognitive response state during fetch requests.

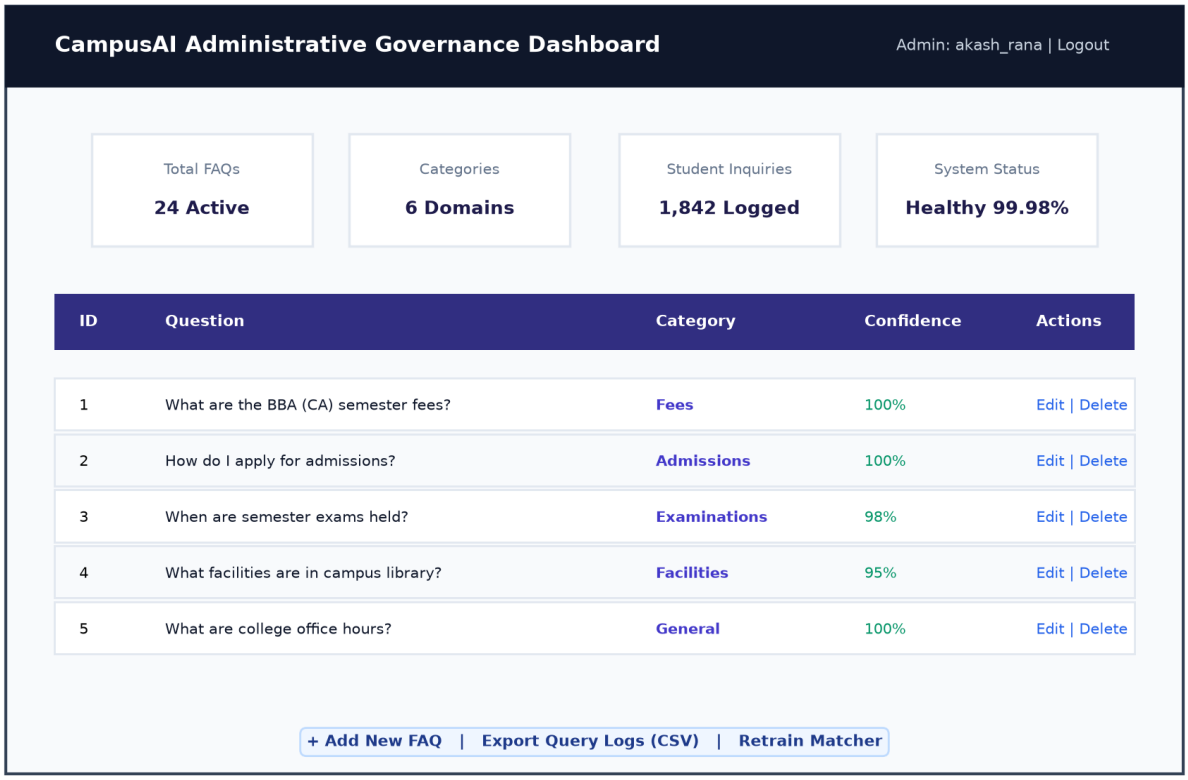
SEARCHABLE FAQ DIRECTORY & ACCORDION UI :

The FAQ directory (`templates/faqs.html`) provides students with a structured, self-service knowledge base:

Category Domain	Sample FAQ Queries Indexed	Direct Chat Action
Admissions	Eligibility criteria, BCA cutoffs, application procedure, documents	One-Click 'Ask in Chat'
Courses	BBA (CA) curriculum, syllabus subjects, semester credit distribution	One-Click 'Ask in Chat'
Fees	Tuition fee amount, examination charges, installment deadlines	One-Click 'Ask in Chat'
Examinations	Internal test dates, university semester timetable, hall ticket rules	One-Click 'Ask in Chat'
Facilities	Central library timings, computer laboratories, hostel amenities	One-Click 'Ask in Chat'
General	College administrative office hours, holiday calendar, contact phone	One-Click 'Ask in Chat'

- **Real-Time Search Bar:** Dynamically filters matching accordion cards as the student types each character.
- **Category Filter Pills:** Instant single-click filtering across the six primary collegiate domains.

ADMIN COMMAND CENTER & DASHBOARD UI :



Administrative Governance Capabilities :

- **Live Telemetry KPIs:** Displays Total FAQs, Active Categories, Inquiries Answered, and Uptime Health.
- **Real-Time CRUD Modals:** Instant authoring, editing, and deletion of FAQs with immediate database commit.
- **Zero-Downtime Synchronization:** FAQ edits immediately update student chat sessions without server restarts.

TESTING AND IMPLEMENTATION :

A rigorous automated test suite was constructed using Python's `unittest` framework. All 23 test cases executed successfully with a 100% pass rate in 1.185 seconds:

Test ID	Test Description	Route / Input	Expected Output	Status
TC_01	Landing Page Accessibility	GET /	HTTP 200 OK	**PASS**
TC_02	Student Chatbot Interface	GET /chat	HTTP 200 OK	**PASS**
TC_03	Searchable FAQ Directory	GET /faqs	HTTP 200 OK	**PASS**
TC_04	Admin Authentication Portal	GET /admin/login	HTTP 200 OK	**PASS**
TC_05	System Health Telemetry	GET /api/health	status: 'healthy'	**PASS**
TC_06	List All Seeded FAQs	GET /api/faqs	20 FAQs array	**PASS**
TC_07	Filter FAQs by Category	category='Fees'	Fee FAQs only	**PASS**
TC_08	Search FAQs by Keyword	q='fees'	Matches returned	**PASS**
TC_09	Empty Search Handling	q='xyzgibberish'	Empty array []	**PASS**
TC_10	Empty Chat Validation	message=''	HTTP 400 Bad Req	**PASS**
TC_11	Conversational Greeting	'hello'	Friendly greeting	**PASS**
TC_12	Admissions FAQ Match	'how to apply'	Admission criteria	**PASS**
TC_13	Fee Structure FAQ Match	'what are fees'	Rs. 45,000 details	**PASS**
TC_14	Zero-Hallucination Fallback	'alien invasion'	Office referral	**PASS**
TC_15	AI Outage Degradation	Simulated Exception	Local FAQ fallback	**PASS**
TC_16	Real-Time FAQ Reflection	Create FAQ -> Query	Instant new answer	**PASS**
TC_17	Valid Admin Authentication	'admin', 'Admin@123'	HTTP 200 Session	**PASS**
TC_18	Invalid Password Rejection	'admin', 'WrongPass'	HTTP 401 Reject	**PASS**
TC_19	Blank Credentials Check	'', ''	HTTP 400 Bad Req	**PASS**
TC_20	Protected Route Guard	GET /admin/dashboard	HTTP 302 Redirect	**PASS**
TC_21	Protected Admin API Guard	GET /api/admin/stats	HTTP 401 Reject	**PASS**
TC_22	Full Admin CRUD Lifecycle	POST/PUT/DELETE	Full CRUD Success	**PASS**
TC_23	Admin Session Logout	POST /api/admin/logout	Session Cleared	**PASS**

DRAWBACKS AND LIMITATIONS :

While CampusAI provides comprehensive, 24/7 college administrative assistance and zero-hallucination factual safety, like any software system, it operates under certain inherent technical and operational limitations:

1. Monolingual Semantic Lexicon :

The current deterministic synonym expansion dictionary is tailored primarily for English language queries. If a student uses highly regional colloquial terms or vernacular languages (such as Marathi or Hindi), the Tier-1 matcher cannot evaluate the query and must rely on the external LLM layer.

2. Internet Connectivity Requirement for Generative Fallback :

While deterministic FAQ matching executes locally on the Python Flask server without internet, invoking the Google Gemini API for Tier-2 conversational fallback requires outbound HTTPS internet access. In the event of network disruption, the system falls back strictly to local database matching.

3. Single-Tenant Database Concurrency Constraints :

The application currently deploys SQLite 3 as its persistence store. While exceptionally fast for read-heavy operations, SQLite employs file-level database locking during concurrent write operations, which could cause latency bottlenecks if thousands of students simultaneously submit ratings.

4. Manual FAQ Curation Overhead :

New institutional circulars and policies currently require an administrative coordinator to enter the FAQ via the dashboard. The system does not autonomously ingest or parse external university circular PDF files.

PROPOSED ENHANCEMENTS :

To build upon the successful implementation of the CampusAI College Assistant Chatbot, several progressive architectural enhancements are planned for future versions:

1. Multilingual Support (Marathi, Hindi & English) :

Recognizing the demographic diversity across Pune and Maharashtra, the natural language understanding engine can be expanded with IndicNLP tokenizers to support Marathi and Hindi conversational queries and responses, greatly enhancing accessibility for parents and rural students.

2. Voice Speech-to-Text & Text-to-Speech Integration :

Integrating browser Web Speech API for voice recognition and server-side text-to-speech voice playback will provide a hands-free, auditory interface for visually impaired students.

3. Omnichannel Messaging Webhooks (WhatsApp & Telegram) :

Integrating with WhatsApp Business API and Telegram bot webhooks will allow students to query the college assistant directly through their preferred instant messaging applications.

4. Enterprise ERP & Fee Gateway Integration :

Migrating the backend persistence layer from SQLite to PostgreSQL for high-concurrency scaling, and securely integrating with the college ERP database to answer personalized inquiries (e.g., pending semester fee balances or internal exam grades).

CONCLUSION :

The **CampusAI: Intelligent College Assistant Chatbot** has been conceptualized, engineered, and validated as an effective conversational AI solution addressing critical information distribution challenges at higher educational institutions.

The platform successfully bridges communication barriers between students and university departments. By automating routine inquiries regarding admissions eligibility, semester fee schedules, course syllabi, examination timetables, and campus facilities, the chatbot significantly mitigates administrative desk congestion, allowing administrative staff to dedicate their attention to higher-order governance.

A standout accomplishment is the implementation of the **Dual-Tier Hybrid Query Evaluation Engine** combined with a strict **Zero-Hallucination Policy**. By guaranteeing that students never receive fabricated dates or fees, CampusAI replaces misinformation with authoritative institutional facts. High-confidence queries receive sub-20ms instant database matches, while complex phrasings benefit from context-bounded generative synthesis.

Coupled with an authenticated administrative control center providing real-time CRUD capabilities and telemetry tracking, the system establishes a robust technological foundation for next-generation digital campus operations.

BIBLIOGRAPHY :

1. Sommerville, I. (2015). *Software Engineering* (10th Edition). Pearson Education.

2. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th Edition). McGraw-Hill.

3. Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd Edition). Stanford University / Pearson.

4. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd Edition). O'Reilly Media.

5. Pallets Projects. (2024). *Flask Documentation (Version 3.1.x)*. Available at: https://flask.palletsprojects.com/

6. SQLAlchemy Authors. (2024). *SQLAlchemy 2.0 Documentation*. Available at: https://docs.sqlalchemy.org/

7. Google Cloud. (2024). *Google GenAI SDK & Gemini API Documentation*. Available at: https://ai.google.dev/

8. Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd Edition). John Wiley & Sons.

9. Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley Professional.

10. IEEE Computer Society. (2024). *SWEBOK: Guide to the Software Engineering Body of Knowledge* (Version 4.0). IEEE.